# tidyfinance: Transparent Factor Construction for Empirical Asset Pricing

*by Christoph Frey, Christoph Scheuch, Stefan Voigt, and Patrick Weiss*

**Abstract** The R package tidyfinance provides research infrastructure for the empirical asset pricing data workflow, which turns raw accounting and return data into factor portfolios for asset pricing tests, portfolio allocation, and the design of trading strategies. The package exposes every construction decision as a configurable parameter with a documented default. A unified download_data() interface covers Wharton Research Data Services (CRSP and Compustat), Ken French's Data Library, the Federal Reserve Economic Data, and other providers, applying provider-specific cleaning. Modular functions automate fiscal-year lagging, breakpoint computation, and portfolio aggregation, so that a default call reproduces standard practice while overriding any default takes a single argument. We illustrate the package by reproducing the asset growth factor of Cooper, Gulen, and Schill (2008) end to end from raw data, and by varying construction choices to show how they move the resulting premium.

## 1 Introduction

Factor returns enter empirical asset pricing tests, optimal portfolio allocation, and risk-management decisions. Constructing them typically involves a three-stage pipeline: data acquisition, usually downloading and cleaning raw return and accounting data from the Center for Research in Security Prices (CRSP) and Compustat; variable construction, including appropriate lags to avoid look-ahead bias; and portfolio sorting, where securities are assigned to portfolios based on characteristics. Each stage involves consequential design choices such as how to handle delistings, which fiscal-year lag to apply, where to set breakpoints, and whether portfolios should be equal- or value-weighted. Few pipelines are standardized, and fewer still ship with executable code. Therefore, reproducing or extending a factor series often starts with reconstructing the pipeline from scratch, with no way to verify that the result matches the original approach.

No widely adopted tool covers this upstream workflow end to end. Ken French's Data Library (Fama and French, 2023), the most popular factor source, publishes returns without the code that produced them. Projects that do publish code (Chen and Zimmermann, 2022; Jensen et al., 2023; Dickerson et al., 2023) release fixed scripts rather than configurable functions, and PyBondLab (Rossetti and Dickerson, 2026) exposes portfolio sorts for corporate bonds but leaves data acquisition and cleaning to the user (Section 2.2). Yet construction choices matter: Walter et al. (2024) and Soebhag et al. (2024) show that they materially change the outcome of portfolio sorts. Existing R packages for quantitative finance work downstream, on price and return panels that the user must already have. Between raw data and standard asset-pricing tests sits an opaque layer in which construction choices accumulate silently.

This paper introduces **tidyfinance**, an R package on CRAN that exposes each stage of the pipeline as configurable code. The package addresses each stage with a dedicated interface. A unified `download_data()` function cleans, aligns, and harmonizes panels from Wharton Research Data Services (WRDS), Ken French's Data Library, Federal Reserve Economic Data (FRED), and 10 other providers, including current constituents for 20 major equity indexes; Table 2 catalogs the full set. The WRDS panels cover CRSP, Compustat, the Trade Reporting and Compliance Engine (TRACE), and the Mergent Fixed Income Securities Database (FISD). Composable helpers then automate fiscal-year lagging, breakpoint computation, and portfolio aggregation. Every construction choice is exposed as a function argument with a default that follows established convention (for instance, value-weights and the Fama-French fiscal-year lag). A default call reproduces standard practice; overriding any default takes one argument.

We illustrate the package by reproducing a version of the asset growth factor of Cooper et al. (2008) end-to-end from raw CRSP and Compustat data, then varying any construction choice with a single-argument change. The package operationalizes the workflow developed in the open-access textbook *Tidy Finance with R* (Scheuch et al., 2023), which presents each step in tutorial form with inline scripts. The contribution of this paper is **tidyfinance** itself: a unified, configurable, tested, and versioned R package for this workflow. Where the book teaches a reader how to assemble the pipeline from scratch, the package supplies it as a single package that researchers can install, call, audit, and reuse without rebuilding the underlying code.

## 2 Background and related work

The R ecosystem for empirical asset pricing splits cleanly into three groups. Each addresses one slice of the workflow and leaves the rest to the user.

**Data-acquisition packages.** Several R packages retrieve data sources common in asset pricing, but none covers the WRDS-CRSP-Compustat panel or the cleaning, alignment, and survivorship conventions the literature requires. **frenchdata** (Areal, 2021) downloads and parses the archives of Ken French's Data Library. **quantmod** (Ryan and Ulrich, 2025) and **tidyquant** (Dancho and Vaughan, 2026) cover price series from Yahoo Finance, FRED, and Tiingo, and **Rblpapi** (Armstrong et al., 2024) queries the Bloomberg terminal. None of them implements a factor-construction step.

**Downstream analysis packages.** Once a cleaned panel is in hand, several packages support analysis. **PerformanceAnalytics** (Peterson and Carl, 2026) computes performance and risk metrics for return series. **fPortfolio** (Wuertz et al., 2023) covers portfolio optimization. **RQuantLib** (Eddelbuettel et al., 2024) interfaces with QuantLib for derivative pricing. **highfrequency** (Boudt et al., 2024) cleans intraday trade-and-quote panels and implements microstructure-volatility estimators. Each of these tools assumes that the panel of returns and factors is already constructed; none implements a factor-construction layer.

**Factor data releases.** A group of projects publishes *factor return series* directly. Ken French's Data Library (Fama and French, 2023) supplies the canonical Fama and French (1993) three- and five-factor returns and dozens of sorted portfolios, but documents construction in prose only and provides no associated code. Akey et al. (2026) show that these series change retroactively as the underlying method is revised, so that the factor returns a researcher obtains depend on *when* they were downloaded, with material effects on mutual-fund performance and cross-sectional asset pricing tests. Three more recent projects pair the published factor data with the construction script that produced it. Hou et al. (2020) replicate 452 anomalies and find that most fail to clear conventional significance once microcaps are down-weighted with NYSE breakpoints and value-weighted returns. The Open Source Cross-Sectional Asset Pricing repository (Chen and Zimmermann, 2022) responds by publishing per-anomaly construction scripts alongside the resulting 212 long-short returns. The Open Source Bond Asset Pricing project (Dickerson et al., 2023) does the same for corporate bond factors. Jensen et al. (2023) provide the Global Factor Data of 153 long-short factors across 93 countries, together with the construction code in Python. These projects let a reader inspect how each factor was built and reproduce its returns, but two limitations remain. First, none of these releases exposes its pipeline as a callable API. A user who wants the asset growth factor rebalanced in January instead of July, or formed on NYSE-plus-NASDAQ breakpoints, or computed on a different fiscal-year lag, must fork the published script and edit it by hand. Second, the factor returns are frozen at the release date, with no in-workflow path to refresh them as new observations arrive or as a researcher's design choices evolve.

**Factor-construction software.** PyBondLab (Rossetti and Dickerson, 2026), a Python package on PyPI since July 2024 and developed alongside the Open Source Bond Asset Pricing project, implements portfolio sorts for corporate bonds as a callable API. Like **tidyfinance**, it offers single and double sorts with staggered rebalancing over overlapping holding periods. Beyond **tidyfinance**, it sorts bonds within each issuer, forms factors on many signals in a single call, and reports portfolio turnover. Its banding option lowers that turnover by keeping a bond in its previous portfolio unless its new portfolio rank differs by a set number of portfolios. **tidyfinance** differs in three respects. First, it integrates data acquisition and cleaning, so that the pipeline starts from raw WRDS panels rather than a user-supplied panel. Second, it targets equities and exposes equity-specific construction choices such as capped value weights and industry and earnings screens. Third, it pairs portfolio sorts with Fama-MacBeth regressions. The two packages thus complement each other across asset classes and languages.

**tidyfinance fills the missing gap:** the package supplies the cleaned WRDS-CRSP-Compustat panel that data-acquisition packages do not produce and exposes the factor-construction pipeline that downstream analysis packages assume to be in hand. One `download_data()` call returns a cleaned WRDS-CRSP-Compustat panel; further functions implement the full factor-construction pipeline (fiscal-year lagging, breakpoint computation, portfolio sorting, long-short aggregation) as a chain of composable steps whose arguments cover the construction choices that vary across studies in the literature above. Two design principles, both inherited from Scheuch et al. (2023), guide the API. First, each construction decision is *transparent*: it appears as a configurable parameter with a documented default, in line with the tidy tools principles of Wickham (2023). Second, the pipeline is *modular* by construction: each construction step is implemented as a narrowly scoped function with clearly defined inputs and outputs. Researchers can therefore either chain these functions into a complete factor-construction workflow or reuse individual components within their own pipelines. **tidyfinance** additionally covers asset pricing tests and optimal portfolio choice; this paper focuses on

factor construction and defers the remaining functionality to dedicated treatments.

## 3 Package overview

### Installation and setup

You can install the released version of **tidyfinance** from CRAN via:

```
install.packages("tidyfinance")
```

To install the development version of **tidyfinance** from GitHub, first install **remotes**, then run:

```
install.packages("remotes")
remotes::install_github("tidy-finance/r-tidyfinance")
```

Once installed, load the package with:

```
library("tidyfinance")
```

### Design and workflow

The empirical asset pricing workflow decomposes into three stages: data acquisition, variable construction, and portfolio sorting. Each stage involves dozens of small decisions: sample filters, cleaning rules, rebalancing conventions, and breakpoint definitions. Table 1 catalogs the user-facing functions exported by **tidyfinance**, grouped by workflow stage. The remainder of this section walks through them in narrative order.

**Table 1:** User-facing functions in **tidyfinance**, grouped by workflow stage. Functions whose first argument is a panel chain naturally via the native pipe `|>`; `filter_options()`, `breakpoint_options()`, `portfolio_sort_options()`, and `data_options()` (explained in detail below) bundle the configurable arguments for the sorting helpers.

| Function | Purpose |
|---|---|
| *Data acquisition* | |
| `download_data()` | Retrieves and cleans a panel from supported data providers. |
| `set_wrds_credentials()` | Stores WRDS username and password in `.Renviron` for subsequent calls. |
| *Variable construction* | |
| `add_lagged_columns()` | Attaches the most recent observed value within a configurable look-back window, with optional Fama-French timing. |
| `join_lagged_values()` | Merges lagged values from a source panel into a target panel using the same windowing logic. |
| `winsorize()` | Replaces values outside the $q$-th and $(1-q)$-th percentiles with the corresponding quantile. |
| `trim()` | Sets values outside the $q$-th and $(1-q)$-th percentiles to `NA`. |
| `compute_rolling_value()` | Applies an aggregation function over a calendar-time rolling window. |
| `filter_sorting_data()` | Applies the sample filters bundled in `filter_options()`. |
| *Portfolio sorting* | |
| `compute_breakpoints()` | Computes quantile cutoffs from a cross-section, optionally restricted by exchange or by minimum market-cap quantile. |
| `assign_portfolio()` | Maps each observation to a portfolio integer using supplied or computed breakpoints. |
| `compute_portfolio_returns()` | Forms portfolios with monthly, annual, or staggered rebalancing and returns equal-, value-, and capped-value-weighted return panels. |
| `compute_long_short_returns()` | Computes the top-minus-bottom (or bottom-minus-top) spread from the output of `compute_portfolio_returns()`. |

| Function | Purpose |
|---|---|
| `implement_portfolio_sort()` | Wraps filtering, breakpoint computation, assignment, and aggregation into a single call for univariate and bivariate sorts. |
| *Asset pricing tests* | |
| `estimate_fama_macbeth()` | Runs cross-sectional ordinary least squares (OLS) regressions each period and aggregates into average risk premia with prewhitened Newey and West (1987) standard errors and the Newey and West (1994) bandwidth. |
| `estimate_betas()` | Estimates rolling-window CAPM betas for each asset. |
| `estimate_model()` | Fits a linear model from a formula string and returns its coefficients as a single-row data frame, one column per coefficient. |

### Data acquisition

**tidyfinance** exposes one interface, `download_data()`, to 332 datasets: equity returns, fixed-income transactions, macroeconomic predictors, index constituents, and more. Table 2 lists all available sources. `list_supported_datasets()` returns the full catalog programmatically.

**Table 2:** Data sources available via `download_data()` in **tidyfinance**. The last column gives example arguments: the `domain` and, where needed, the `dataset` or another selecting argument, e.g., `download_data("WRDS", "crsp_monthly")`. For every dataset with a time dimension, the same `start_date` and `end_date` arguments subset the returned panel to a user-specified window. Freq. lists the available data frequencies: D = daily, W = weekly, M = monthly, Q = quarterly, A = annual, I = intraday (trade-level or 5-second snapshots), D–A = native frequency of each series, – = no time dimension.

| Source | Description | Freq. | `download_data()` arguments |
|---|---|---|---|
| *Public data sources* | | | |
| Ken French's Data Library | Fama-French 297 factor models, portfolios, industry classifications | D/W/M | `"Fama-French", "Fama/French 3 Factors"` |
| Global Q | Hou et al. (2021) q5 factor model (market, size, investment, ROE, expected growth) | D/W/M/Q/A | `"Global Q", "factors_q5_monthly"` |
| Stambaugh-Yuan | Stambaugh and Yuan (2017) mispricing factors (MGMT, PERF) | D/M | `"Stambaugh-Yuan", "monthly"` |
| Pastor-Stambaugh | Pástor and Stambaugh (2003) traded and non-traded liquidity factors | M | `"Pastor-Stambaugh"` |
| Global Factor Data | Jensen et al. (2023) 153 long-short factors across 93 countries | D/M | `"Global Factor Data", "factors"` |
| Goyal-Welch Predictors | Macroeconomic return predictors | M/Q/A | `"Goyal-Welch", "monthly"` |
| FRED | St. Louis Fed economic time series (GDP, CPI, etc.) | D–A | `"FRED", series = "GDP"` |
| FRED-MD and FRED-QD | Macroeconomic panels (McCracken and Ng, 2016, 2021), including real-time vintages | M/Q | `"FRED", "FRED-MD"` |
| Damodaran | Damodaran (2026) cost of capital by industry and region | A | `"Damodaran", "wacc"` |
| Yahoo Finance | Historical price data with adjusted close for any ticker | D | `"Stock Prices", symbols = "AAPL"` |
| Index Constituents | Current members of major equity indexes (S&P 500, Russell, etc.) | – | `"Index Constituents", index = "DAX"` |
| Open Source Asset Pricing | 212 monthly asset pricing signals and anomalies | M | `"Open Source Asset Pricing"` |

| Source | Description | Freq. | `download_data()` arguments |
|---|---|---|---|
| High-Frequency | Aggregated order book snapshots for S&P 500 constituents constructed by Hautsch et al. (2026) | I | `"Tidy Finance"`, `"high_frequency_sp500"` |
| Risk-Free Rate | Risk-free rates derived from FRED T-bill series by **tidyfinance** | D/M | `"Tidy Finance"`, `"risk_free"` |
| Factor Library | Precomputed factor returns hosted on Hugging Face Hub | M | `"Tidy Finance"`, `"factor_library"` |
| *WRDS (subscription required)* | | | |
| CRSP | Stock returns, market cap, listing exchange, industry information | D/M | `"WRDS"`, `"crsp_monthly"` |
| Compustat | Financial statements | Q/A | `"WRDS"`, `"compustat_annual"` |
| CCM Links | CRSP-Compustat linking table (`permno` to `gvkey`) | – | `"WRDS"`, `"ccm_links"` |
| FISD | Mergent Fixed Income Securities Database | – | `"WRDS"`, `"fisd"` |
| TRACE | Corporate bond transactions Dick-Nielsen (2009) cleaned | I | `"WRDS"`, `"trace_enhanced"` |

A first illustration uses only public data. A single call downloads and parses the Fama-French three-factor data from Ken French's Data Library:

```
ff_factors <- download_data(
  domain = "Fama-French",
  dataset = "Fama/French 3 Factors",
  start_date = "1972-01-01",
  end_date = "2024-12-31"
)
```

The `start_date` and `end_date` arguments generalize across every dataset type in Table 2 that supports a time dimension.

Public data still requires nontrivial cleaning. With `domain = "Goyal-Welch"`, `download_data()` parses the raw spreadsheet of the Welch and Goyal (2008) predictors at monthly, quarterly, or annual frequency (set via `dataset`) and constructs the standard equity-premium predictors, such as the log dividend-price ratio (`dp`), the term spread (`tms`), and the default-yield spread (`dfy`).

For empirical asset pricing work, WRDS is the common source of return and accounting panels. WRDS data are proprietary and require an institutional subscription and credentials. **tidyfinance** handles this setup once. `set_wrds_credentials()` prompts for the username and password and stores them in a project-specific or global `.Renviron` file, and prompts the user to add `.Renviron` to `.gitignore` to prevent credential leakage. Subsequent `download_data()` calls open the database connection automatically. Credentials alone do not suffice: before the first connection, users must enroll in WRDS two-factor authentication (2FA) and accept the WRDS terms of use on the WRDS website. Each connection then triggers a 2FA push notification to the registered device; once approved, the connection proceeds normally. The help page `?get_wrds_connection` lists the full setup checklist.

`download_data()` applies standard cleaning conventions following Bali et al. (2016) and Scheuch et al. (2023) automatically. For CRSP monthly data, the function restricts the universe to common equity securities listed on NYSE, AMEX (now NYSE American), or NASDAQ. It computes market capitalization as shares outstanding times price, constructs a one-month market capitalization lag for portfolio weights, and maps listing exchange codes and Standard Industrial Classification (SIC) codes to readable labels. Excess returns are raw returns minus a risk-free rate that **tidyfinance** constructs from FRED by splicing the 3-month Treasury bill secondary-market rate (pre-2001) with the 4-week Treasury bill secondary-market rate (from 2001 onward); the spliced series is hosted on Hugging Face and refreshed monthly by a scheduled GitHub Actions workflow. The function drops observations with missing returns or market capitalization, and incorporates delisting returns directly, eliminating the need for a separate adjustment. The function defaults to the CRSP 2.0 (CIZ) layout first introduced in July 2022. Alternatively, passing `version = "v1"` returns the legacy (SIZ) layout, which ends in December 2024. Schwarz et al. (2026) document that the move to CIZ rewrites a non-trivial share of historical monthly returns through a changed dividend reinvestment assumption, yet find that average premia and their significance are largely unaffected.

For Compustat annual data, `download_data()` restricts to industrial-format consolidated statements and retains only the most recent observation per firm-year. Following Fama and French (1992),

it constructs book equity in priority order: stockholders' equity when available, else common equity plus preferred stock, else total assets minus total liabilities. The function adds back deferred taxes and investment tax credits, and subtracts the book value of preferred stock.

For TRACE corporate bond transactions, `download_data()` applies the Dick-Nielsen cleaning protocol (Dick-Nielsen, 2009) to remove cancellations, corrections, and reversals before returning the trade panel.

Across `download_data()`, every returned panel exposes a single `date` column of `<date>` type, regardless of source. Joining observations from different providers therefore reduces to an equality join on `date` without manual format conversion. The convention is the same across frequencies: daily observations return the trading date, monthly observations the first calendar day of the reference month (regardless of whether it was a trading day), and quarterly and annual observations the first day of the fiscal period-end month as reported by the original source. For Compustat, the latter is the firm's fiscal period-end (`datadate`). Macroeconomic series from FRED keep their original release dates. Where the underlying source uses an internal date that differs from this convention, **tidyfinance** preserves it as an auxiliary column: CRSP 2.0 monthly returns, for example, retain the trading-date `mthcaldt` as `calculation_date` alongside the harmonized `date`. Cross-frequency joins, such as merging monthly returns with annual fundamentals, require the lagging logic described in Section 2.3.4.

### Variable construction

Most predictive variables in asset pricing studies use lagged information so that, for example, portfolio formation in month $t$ uses only data observable at the end of month $t-1$. `add_lagged_columns()` automates the required lagging procedure. For each row in the data, it looks back over a configurable window and attaches the most recent observed value of the chosen columns. Figure 1 illustrates the two modes the function supports. By default (`ff_adjustment = FALSE`), shown in Panel A, the window is bounded by `lag` (the minimum delay between observation and use) and `max_lag` (the maximum delay before a stale value is treated as missing). Within this window, the function picks the most recent value, skipping missing values if `drop_na = TRUE`, as needed for monthly panels whose accounting columns are only filled at fiscal year-ends; if no observation falls in the window, the lagged column takes `NA`.

The second mode, enabled by `ff_adjustment = TRUE` and shown in Panel B of Figure 1, implements the Fama-French convention (Fama and French, 1993). The convention assumes accounting variables for fiscal year $t-1$ become public information by June of year $t$, then uses them for portfolio formation from July of $t$ through June of $t+1$. This timing produces a step-function lag that updates once per year at the June/July boundary, regardless of the underlying fiscal year-end, and ignores `lag` and `max_lag`. Figure 1 contrasts the two modes: with a 7-month lag, a firm with a June year-end updates in January, but under the convention it updates in July, like a firm with a December year-end. If a fiscal year is missing, the lag window drops the old value 18 months after its fiscal year-end, while the convention drops it at the next July.

`join_lagged_values()` applies the same logic, including `ff_adjustment`, when it merges lagged values from one data frame into another with a different frequency, for example, annual accounting data into a monthly return panel.

A handful of additional helpers cover routine operations that recur across empirical studies. `winsorize()` and `trim()` handle outliers in cross-sectional variables: the former symmetrically replaces values below the $q$-th and above the $(1-q)$-th percentile with the corresponding quantile values, while the latter sets them to `NA`. `estimate_model()` fits a linear model from a formula string and returns the coefficients as a tidy one-row data frame, which composes naturally with `group_by()` for grouped or rolling regressions such as estimating time-series factor exposures stock by stock via `estimate_betas()`. `compute_rolling_value()` applies an arbitrary aggregation function over a rolling window defined in calendar time rather than row counts, accommodating irregular observation frequencies and missing periods. `filter_sorting_data()` screens a panel before sorting, dropping observations that fail the sample filters bundled in a `filter_options()` object, such as financial or utility firms or stocks below a minimum price or size; by default it reports how many observations each filter removes, which `quiet = TRUE` suppresses.

### Portfolio sorting

A portfolio sort groups stocks each period into ordered portfolios according to the level of a sorting variable. The boundaries between portfolios are called *breakpoints*: numeric cutoffs of the sorting variable that determine which portfolio a stock joins. A quintile sort on size, for instance, has four

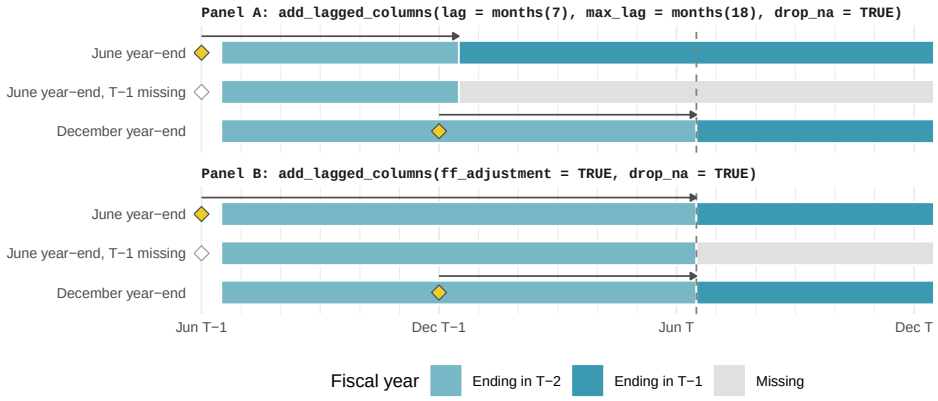



**Figure 1:** Lagging modes of add_lagged_columns() for firms with a June or a December fiscal year-end and for a June firm whose fiscal year T-1 is missing. Bars show which fiscal year's value each month receives; diamonds mark the fiscal year-ends in year T-1 (hollow if missing), and arrows point to the first month that uses them. Panel A uses a 7-to-18-month lag window, so each firm updates seven months after its own fiscal year-end; Panel B uses the Fama-French convention, so every firm updates in July.

breakpoints at the 20th, 40th, 60th, and 80th percentiles of the cross-sectional market-capitalization distribution. Common sorting variables include size (measured as market capitalization), beta, book-to-market, and momentum. The long-short return between the extreme portfolios is interpreted as a factor-mimicking strategy and serves as the basis for tests of whether the sorting variable forecasts future returns. Portfolio sorts are among the most widely used tools in empirical asset pricing (Bali et al., 2016). We expose the procedure through four composable building blocks plus a one-call wrapper.

`compute_breakpoints()` takes a cross-section and a sorting variable and returns the numeric cutoffs that define the portfolio boundaries. The user supplies either a number of portfolios (for example, five for quintiles) or an explicit vector of percentiles, but not both. A `breakpoints_exchanges` argument restricts the universe used to compute breakpoints. The standard convention is NYSE-listed stocks only, which prevents the large cross-section of small NASDAQ-listed firms from dominating the breakpoints (Fama and French, 1993). A related `breakpoints_min_size_threshold` argument excludes stocks below a market-cap quantile from the breakpoint computation, which size-related sorts often require because microcaps would otherwise distort the cutoffs. A `smooth_bunching` flag handles cases where the sorting variable takes the same value for many observations at the bottom or top of the distribution, so that the lowest (or highest) quantile breakpoints coincide. Without intervention, all tied observations would land in a single boundary portfolio while neighboring portfolios remain empty. When `smooth_bunching = TRUE`, the function assigns the bunched observations to the boundary portfolio and recomputes the remaining breakpoints on the subset of observations strictly inside the boundary value, restoring well-defined cutoffs for the interior portfolios.

`assign_portfolio()` maps each observation to a portfolio integer and uses breakpoints supplied either directly or via a breakpoint-computing function (defaulting to `compute_breakpoints()`). In practice, `assign_portfolio()` is called separately for each rebalancing date, so that portfolios are reformed each period rather than fixed once for the whole sample.

`compute_portfolio_returns()` ties the lower-level pieces together. Given a panel of stock data, one or two sorting variables, a sorting method, and `breakpoint_options`, it handles rebalancing, breakpoint computation, portfolio assignment, and return aggregation. It then returns a complete panel of portfolio returns. For each portfolio-date the function reports three return measures: equal-weighted, value-weighted using `mktcap_lag` as the weight, and capped value-weighted (Jensen et al., 2023). Following Jensen et al. (2023), the capped version caps each stock's market capitalization at the `cap_weight` percentile of NYSE stocks on each date (default `0.8`, the 80th percentile), which prevents a few megacap firms from dominating the portfolio. A `min_portfolio_size` argument sets the minimum number of constituents below which the corresponding return is set to `NA`. An optional `rebalancing_month` argument switches from period-by-period rebalancing to annual rebalancing in the specified month. Portfolios formed in that month are then held constant for the following twelve months. Alternatively, `holding_period` and `skip` implement staggered rebalancing: portfolios are formed every period, and each cohort is held for `holding_period` periods, starting `skip` periods after formation, so that overlapping cohorts are averaged as in Jegadeesh and Titman (1993).

The function supports three sorting schemes, illustrated in Figure 2. A *univariate* sort uses a single

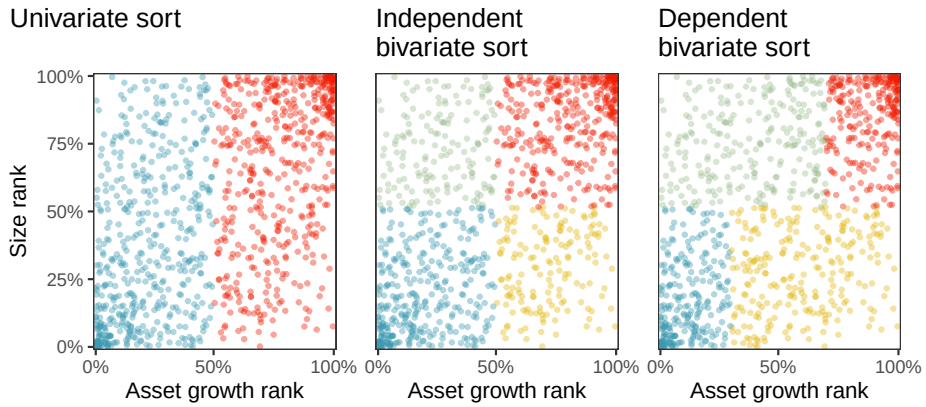



**Figure 2:** Portfolio sort approaches illustrated on a simulated cross-section. Each panel plots simulated observations assigned to 2 × 2 portfolios based on asset growth rank (x-axis) and size rank (y-axis), both expressed as percentiles. Colors indicate portfolio cell assignment. The univariate sort (left) assigns portfolios on asset growth alone; the independent bivariate sort (center) assigns each dimension independently; the dependent bivariate sort (right) sorts on size first, then on asset growth within size groups.

variable. An *independent bivariate* sort computes breakpoints on each variable separately and intersects the resulting bins to produce a two-way grid. A *dependent bivariate* sort first partitions the cross-section by the secondary (control) variable and then re-sorts within each bucket on the main variable, which is the appropriate choice when the second variable is treated as a control rather than as an independent dimension of variation. For bivariate sorts, returns are averaged over the secondary variable so that only the main-variable portfolios are returned.

`compute_long_short_returns()` takes the output of `compute_portfolio_returns()` and computes the spread between the top and bottom portfolios. A `direction` argument controls the sign and defaults to `"top_minus_bottom"`. The function returns one long-short series per return measure available in the input, so the equal-weighted, value-weighted, and capped value-weighted spreads are produced together. This long-short return is the factor-mimicking spread that the asset-pricing literature typically takes as the object of interest.

For users who want the full pipeline in a single call, `implement_portfolio_sort()` wraps the four functions above. It takes a panel of stock data, one or two sorting variables, a sorting method, and a configuration object, and returns the panel of portfolio returns; piping that result to `compute_long_short_returns()` yields the long-short factor series, as in the case study below. The lower-level functions remain available for custom workflows that need finer control over breakpoints, weighting, or rebalancing.

## 4 A case study in factor construction

We walk through the construction of the asset growth factor from raw CRSP and Compustat data to long-short portfolio returns, applying each stage of Table 1 in turn.

Load **tidyfinance** together with a minimal set of supporting packages: **dplyr** (Wickham et al., 2026) for data manipulation, **tidyr** (Wickham et al., 2024) for reshaping between long and wide formats, and **lubridate** (Spinu et al., 2024) for date arithmetic.

```
library("tidyfinance")
library("dplyr")
library("tidyr")
library("lubridate")
```

### Data acquisition

Constructing the asset growth factor requires two WRDS panels: annual accounting data from Compustat to compute the asset growth signal and monthly stock returns from CRSP to translate the signal into portfolio returns.

CRSP and Compustat are proprietary and require a WRDS subscription. All numbers, tables, and figures in this paper are computed from the real WRDS data with `data_domain <- "WRDS"`.

```
data_domain <- "WRDS"
```

Readers without a WRDS subscription can render the entire manuscript by setting `data_domain <- "Pseudo Data"` in the chunk above. Every `download_data()` call for the three WRDS datasets of the case study (monthly CRSP returns, annual Compustat fundamentals, and the CRSP-Compustat links) then returns simulated data from an in-package generator. The pseudo data keep the column names, data types, missingness patterns, and panel structure of the real data, so the cleaning, lagging, and sorting code runs through the same branches and every table and figure renders. They do not keep economic relations: returns are independent draws, so the long-short premia are noise. The `seed` argument makes the draws reproducible, and `n_assets` sets the size of the cross-section (default `1000L`). With WRDS access, `set_wrds_credentials()` stores the credentials once (Section 2.3.3).

```
date_start <- as.Date("1972-01-01")
date_end <- as.Date("2024-12-31")
```

We restrict the sample to the period from 1972 onward, before which Compustat's coverage of the cross-section is sparser, and we end it in December 2024.

First, we download monthly stock returns from CRSP. Setting `add_ccm_links = TRUE` appends CRSP-Compustat linking identifiers needed to merge with financial statement data in the next step.

```
crsp_monthly <- download_data(
  domain = data_domain,
  dataset = "crsp_monthly",
  version = "v2",
  start_date = date_start,
  end_date = date_end,
  add_ccm_links = TRUE,
  quiet = TRUE
)
```

The dataset contains 3,195,812 stock-month observations for 25,369 unique stocks, covering the period from January 1st, 1972 to December 31st, 2024.

Next, we download annual financial statements from Compustat.

```
compustat_annual <- download_data(
  domain = data_domain,
  dataset = "compustat_annual",
  start_date = date_start,
  end_date = date_end,
  additional_columns = c("at", "ib"),
  only_usd = TRUE,
  quiet = TRUE
) |>
  select(gvkey, date, at, ib)
```

Annual Compustat data contain 472,169 firm-year observations for 39,392 unique firms, covering the period from January 1st, 1972 to December 31st, 2024. The `additional_columns` argument retrieves total assets (`at`) and income before extraordinary items (`ib`), needed for computing asset growth and applying an earnings filter in later steps. Setting `only_usd = TRUE` restricts the dataset to USD-denominated stocks.

### Variable construction

We illustrate data cleaning and variable construction using asset growth (AG) as our running example. Asset growth for a firm $i$ in year $t$ is the year-over-year percentage change in total assets (Cooper et al., 2008),

$$AG_{t,i} = (AT_{t,i} - AT_{t-1,i})/AT_{t-1,i},$$

where $AT_{t,i}$ is total assets for firm $i$ in year $t$ (retrieved in column `at`).

The `add_lagged_columns()` function from **tidyfinance** handles temporal alignment by creating a lagged version of total assets grouped by firm identifier (`gvkey`) with a one-year lag. Non-finite values (arising when lagged total assets are zero or missing) become `NA`.

**Table 3:** Cross-sectional summary statistics for asset growth (winsorized at the 1st and 99th percentiles). The 'Pooled' column reports the statistic computed on the entire firm-year sample. The 'Yearly' columns summarize the time series of statistic values computed cross-sectionally year by year, taking the mean, minimum, and maximum across calendar years. N is the count of firm-year observations (pooled) or per-year observations (yearly columns). Sample: 1973–2024.

| Statistic | Pooled | Yearly Mean | Yearly Min | Yearly Max |
|---|---|---|---|---|
| N | 358,055 | 6,886 | 3,808 | 9,467 |
| Mean | 0.249 | 0.261 | 0.048 | 0.889 |
| SD | 0.933 | 0.982 | 0.203 | 3.830 |
| P10 | -0.205 | -0.196 | -0.397 | -0.007 |
| P25 | -0.041 | -0.039 | -0.151 | 0.060 |
| Median | 0.061 | 0.062 | 0.001 | 0.132 |
| P75 | 0.208 | 0.210 | 0.105 | 0.370 |
| P90 | 0.609 | 0.631 | 0.253 | 1.341 |

```
sorting_variable_data <- compustat_annual |>
  add_lagged_columns(
    cols = "at",
    lag = years(1),
    by = "gvkey"
  ) |>
  mutate(
    asset_growth = (at - at_lag) / at_lag,
    asset_growth = if_else(is.finite(asset_growth), asset_growth, NA_real_)
  ) |>
  select(gvkey, date, asset_growth, ib)
```

The call relies on two silent defaults: `max_lag` falls back to `lag` (here one year), so only the single observation exactly one year before each row is eligible; `ff_adjustment = FALSE`, so the look-back is the plain calendar window rather than the Fama-French June/July rebalancing convention introduced in Section 2.3.4.

`sorting_variable_data` contains one observation per firm-year, the natural unit for cross-sectional analysis of annual accounting variables. Asset growth is heavily right-skewed: a small number of firms with dramatic balance-sheet expansions generate extreme positive values. Table 3 reports the pooled cross-sectional summary alongside the time-series average, minimum, and maximum of the corresponding statistic computed cross-sectionally each year.[1] We obtain both views with `create_summary_statistics()`, a **tidyfinance** helper that returns the standard set of cross-sectional summary statistics for a variable: applied to the pooled (winsorized) sample for the first column and grouped by calendar year for the next three. Table 3 assembles the two computations side by side.

```
pooled_stats <- sorting_variable_data |>
  drop_na(asset_growth) |>
  mutate(asset_growth = winsorize(asset_growth, 0.01)) |>
  create_summary_statistics(asset_growth, detail = TRUE)

yearly_stats <- sorting_variable_data |>
  drop_na(asset_growth) |>
  group_by(year = year(date)) |>
  mutate(asset_growth = winsorize(asset_growth, 0.01)) |>
  ungroup() |>
  create_summary_statistics(asset_growth, by = year, detail = TRUE)
```

The next step uses `join_lagged_values()` to merge the annual sorting variables onto the monthly CRSP panel. Setting `ff_adjustment = TRUE` implements the Fama-French convention (Fama and French, 1993): the accounting data of a fiscal year ending in calendar year $t$ are used from July of $t+1$ through June of $t+2$, whatever the month of the fiscal year-end. Every firm therefore updates in July, and no lag window is needed.

[1]The pooled count of 358,055 firm-years is below the 472,169 downloaded in Section 2.4.1 because asset growth needs total assets in both the current fiscal year and the one exactly twelve months earlier; firm-years with missing total assets, each firm's first year, and gaps or changes in fiscal year-end all drop out.

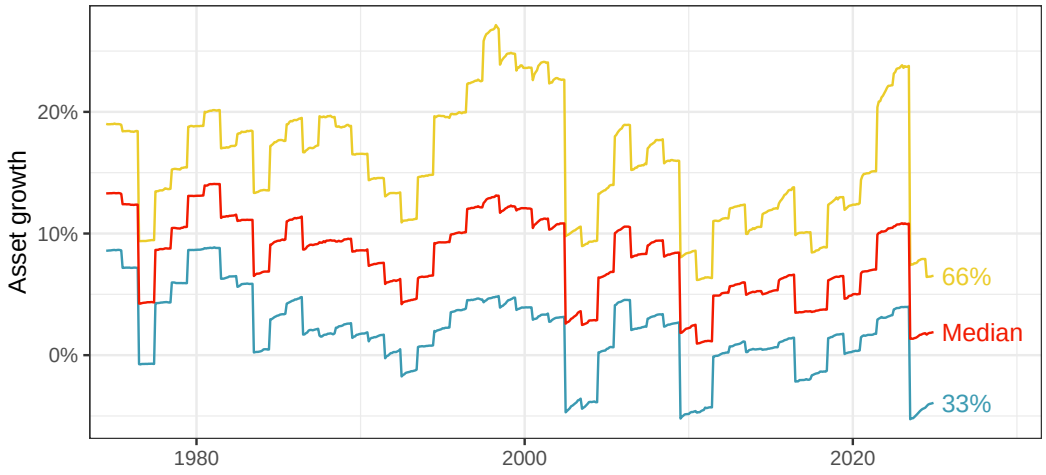



**Figure 3:** Selected percentiles (33rd, 50th, 66th) of cross-sectional asset growth over time. The y-axis reports annual growth in total assets in percent. Sample: 1974–2024.

```
sorting_data <- crsp_monthly |>
  join_lagged_values(
    new_data = sorting_variable_data,
    id_keys = "gvkey",
    ff_adjustment = TRUE
  ) |>
  select(
    date,
    permno,
    exchange,
    siccd,
    ret_excess,
    mktcap_lag,
    asset_growth,
    ib
  ) |>
  filter(date >= date_start + months(12) + months(18))
```

The filter drops the first year, which only serves to compute growth rates, and the following 18 months, in which the lagged signal is not yet available. The columns `siccd` and `ib` support the sample filters in Section 2.4.3.

Figure 3 shows the 33rd, 50th, and 66th cross-sectional percentiles of asset growth over time in the resulting panel; both the median and the dispersion across firms vary substantially over the sample.

With the sorting variable constructed and merged onto the monthly stock panel, we now turn to portfolio sorts, the mechanism that translates firm-level characteristics into tradable factor returns.

### Portfolio sorting

In portfolio sorts, stocks are ranked each period by their sorting variable and divided into quantile portfolios. Portfolio returns are equal-weighted or value-weighted averages of constituent stock returns. The long-short factor return is the difference between the extreme portfolios. For asset growth, the relevant spread is bottom minus top, reflecting the negative premium documented by Cooper et al. (2008). The construction involves several methodological choices: which stocks enter the universe, how many portfolios to form, which exchange determines the breakpoints, the weighting scheme, and the rebalancing frequency.

`implement_portfolio_sort()` exposes all of these choices through a single function call. The function takes the panel data, one or two sorting variables, a sorting method, and a `portfolio_sort_options` object that bundles two configuration layers: `filter_options()` controls which observations enter the breakpoint and return computation (for example, excluding financial firms or stocks below a minimum price), and `breakpoint_options()` controls how portfolios are formed (number of portfolios, exchange universe, bunching). Internally, the function applies the sample filters and then calls

`compute_portfolio_returns()`, which produces the time series of portfolio returns. A subsequent call to `compute_long_short_returns()` constructs the long-short portfolio.

`compute_portfolio_returns()` itself rests on two lower-level helpers, `compute_breakpoints()` for quantile cutoffs and `assign_portfolio()` for mapping each stock-month to a portfolio number. Both are exported, so researchers who need finer control can call them directly rather than through the wrapper. The two examples below use `implement_portfolio_sort()` throughout.

### Baseline specification (univariate sorts)

The simplest call to `implement_portfolio_sort()` requires only a `portfolio_sort_options` object that specifies how to form the breakpoints. In a univariate sort, stocks are ranked on a single characteristic. We illustrate using asset growth as the sorting variable, forming quintile portfolios with breakpoints computed from NYSE-listed stocks only, and rebalancing annually in July.

```
options_baseline <- portfolio_sort_options(
  breakpoint_options_main = breakpoint_options(
    n_portfolios = 5,
    breakpoints_exchanges = "NYSE"
  )
)

portfolio_returns <- implement_portfolio_sort(
  data = sorting_data,
  sorting_variables = "asset_growth",
  sorting_method = "univariate",
  rebalancing_month = 7,
  portfolio_sort_options = options_baseline
)
```

Apart from setting five portfolios, NYSE breakpoints, and `rebalancing_month = 7`, every other choice point falls back to the package default: a cap-weight cutoff at the 80th percentile of NYSE market capitalization for the capped value-weighted return, no minimum portfolio size, and no smooth bunching. Setting `rebalancing_month = 7` reforms the portfolios each July and holds them for twelve months, the month in which `ff_adjustment = TRUE` makes a new fiscal-year value available.

`implement_portfolio_sort()` returns one row per portfolio-month with the value-weighted (VW), equal-weighted (EW), and capped value-weighted (VWC) returns of Section 2.3.5. With `direction = "bottom_minus_top"`, `compute_long_short_returns()` turns them into the factor, long low-AG and short high-AG stocks.

```
factor_returns_ag <- portfolio_returns |>
  compute_long_short_returns(direction = "bottom_minus_top")
```

Table 4 summarizes the time series of each quintile across all three weighting schemes, with the long-short portfolio appended as the final row.

```
return_summary <- function(data) {
  data |>
    drop_na() |>
    summarize(
      across(
        c(ret_excess_vw, ret_excess_ew, ret_excess_vw_capped),
        list(
          mean = \(x) mean(x) * 12,
          tstat = \(x) mean(x) / (sd(x) / sqrt(n()))
        ),
        .names = "{.col}_{.fn}"
      ),
      .groups = "drop"
    )
}
univariate_sort_table <- bind_rows(
  portfolio_returns |>
```

**Table 4:** Time-series averages of annualized monthly excess returns for asset growth quintile portfolios. Portfolios are formed annually in July using NYSE breakpoints. Means are annualized (multiplied by 12). The $t$ statistic tests whether the mean monthly excess return differs from zero. Capped value-weighted caps market capitalization at the 80th percentile of NYSE stocks. The L/S row is the long-short portfolio (bottom minus top quintile). Sample: 1974–2024.

| | Value-weighted | | Equal-weighted | | Capped value-weighted | |
|---|---|---|---|---|---|---|
| Portfolio | Mean | $t$ statistic | Mean | $t$ statistic | Mean | $t$ statistic |
| 1 | 0.110 | 4.586 | 0.148 | 4.299 | 0.113 | 4.302 |
| 2 | 0.094 | 4.600 | 0.129 | 5.128 | 0.104 | 4.627 |
| 3 | 0.085 | 4.034 | 0.122 | 4.986 | 0.103 | 4.594 |
| 4 | 0.086 | 3.804 | 0.114 | 4.373 | 0.098 | 4.090 |
| 5 | 0.075 | 2.675 | 0.055 | 1.651 | 0.069 | 2.306 |
| L/S | 0.035 | 2.321 | 0.093 | 7.583 | 0.043 | 3.451 |

```
    group_by(portfolio) |>
    return_summary() |>
    mutate(portfolio = as.character(portfolio)),
  factor_returns_ag |>
    return_summary() |>
    mutate(portfolio = "L/S")
)
```

The value-weighted long-short portfolio earns 3.5% per year ($t$-statistic of 2.32). The sign matches Cooper et al. (2008); we compare magnitudes in Section 2.4.3. The equal-weighted spread is considerably larger, which indicates that the effect is stronger among the small stocks that equal weighting emphasizes. Mean returns decline from the lowest to the highest AG quintile under all three weighting schemes, monotonically for equal and capped value weights and with nearly equal third and fourth quintiles for value weights. A single function call thus recovers the negative relation between asset growth and subsequent returns from raw WRDS data.

### Customized specification (bivariate sorts)

The baseline specification above takes nearly every package default and runs a univariate sort to form long-short portfolios. The customized specification below changes both: it overrides several of those defaults through `portfolio_sort_options` and moves from sorting on one variable to sorting on two.

We adopt a specification that combines, in a single call to `implement_portfolio_sort()`, the universe restrictions and breakpoint conventions used in many empirical studies: exclude financials and utilities, exclude firms with non-positive earnings, ignore the smallest 20% of NYSE-listed firms when computing breakpoints, form quintiles on AG within size buckets, use NYSE-only breakpoints, rebalance annually in July, and require at least ten constituents per portfolio.

Bivariate sorts control for a second variable, typically firm size (`mktcap_lag`), when forming portfolios on the main characteristic. We use the dependent variant introduced in Section 2.3.5 and illustrated in Figure 2: stocks are first split into size buckets, then sorted into asset growth portfolios within each bucket, with returns averaged across the size dimension.

```
options_customized <- portfolio_sort_options(
  filter_options = filter_options(
    exclude_financials = TRUE,
    exclude_utilities = TRUE,
    exclude_negative_earnings = TRUE
  ),
  breakpoint_options_main = breakpoint_options(
    n_portfolios = 5,
    breakpoints_exchanges = "NYSE",
    breakpoints_min_size_threshold = 0.2
  ),
  breakpoint_options_secondary = breakpoint_options(
    n_portfolios = 2,
    breakpoints_exchanges = "NYSE"
  )
```

```
)

portfolio_returns_customized <- implement_portfolio_sort(
  data = sorting_data,
  sorting_variables = c("asset_growth", "mktcap_lag"),
  sorting_method = "bivariate-dependent",
  rebalancing_month = 7,
  portfolio_sort_options = options_customized,
  min_portfolio_size = 10,
  quiet = TRUE
)
factor_returns_customized <- portfolio_returns_customized |>
  compute_long_short_returns(direction = "bottom_minus_top")
```

A single options object (`options_customized`) bundles every methodological choice that distinguishes this specification from the baseline; nothing in the surrounding pipeline changes. Setting `quiet = TRUE` suppresses the messages that report how many observations each filter drops. With annual rebalancing, the screens apply in the formation month only, so a stock that, for example, reports a loss after formation keeps its returns until the portfolios are reformed. `filter_options(timing = "every_period")` applies the screens in every month instead, but this uses information that is not available at formation and the package warns about it.

For comparison, we also include a bivariate-dependent sort that uses the default filters and breakpoint conventions, so the only difference from the baseline is the addition of size control. The long-short returns from that specification are appended to the customized one in Table 5.

```
options_bivariate <- portfolio_sort_options(
  breakpoint_options_main = breakpoint_options(
    n_portfolios = 5,
    breakpoints_exchanges = "NYSE"
  ),
  breakpoint_options_secondary = breakpoint_options(
    n_portfolios = 2,
    breakpoints_exchanges = "NYSE"
  )
)
portfolio_returns_bivariate <- implement_portfolio_sort(
  data = sorting_data,
  sorting_variables = c("asset_growth", "mktcap_lag"),
  sorting_method = "bivariate-dependent",
  rebalancing_month = 7,
  portfolio_sort_options = options_bivariate
)
factor_returns_bivariate <- portfolio_returns_bivariate |>
  compute_long_short_returns(direction = "bottom_minus_top")
```

Table 5 also reports two sensitivity checks of the baseline, each a one-argument change. The first drops `ff_adjustment` and uses a plain lag window instead: each month receives the most recent fiscal-year value observed 7 to 18 months earlier, so each firm updates seven months after its own fiscal year-end rather than in July. The second caps market capitalization at the 80th percentile of all stocks instead of NYSE stocks, which only affects the capped value-weighted returns.

```
sorting_data_window <- crsp_monthly |>
  join_lagged_values(
    new_data = sorting_variable_data,
    id_keys = "gvkey",
    min_lag = months(7),
    max_lag = months(18)
  ) |>
  filter(date >= date_start + months(12) + months(18))

factor_returns_window <- implement_portfolio_sort(
  data = sorting_data_window,
  sorting_variables = "asset_growth",
```

**Table 5:** Comparison of long-short asset growth returns across three specifications and two sensitivity checks of the baseline. The baseline is a univariate sort with NYSE breakpoints and no filters. The bivariate-only specification adds a dependent bivariate sort on size to the baseline but otherwise keeps the same universe and breakpoints. The customized specification adds the filter set (financials, utilities, non-positive earnings) and a 20% size threshold on the breakpoints. The last two rows change one choice of the baseline: a 7-to-18-month lag window instead of the Fama-French timing, and a cap at the 80th percentile of all stocks instead of NYSE stocks. All rows use annual July rebalancing; means are annualized value-weighted (VW) and capped value-weighted (VWC) excess returns. Sample: 1974–2024.

| Specification | VW mean | VW $t$ | VWC mean | VWC $t$ |
|---|---|---|---|---|
| Baseline (univariate, no filters) | 0.035 | 2.321 | 0.043 | 3.451 |
| Bivariate only (size-controlled, no filters) | 0.036 | 3.064 | 0.039 | 3.437 |
| Customized (size-controlled, filtered) | 0.026 | 1.892 | 0.033 | 2.667 |
| Baseline, 7-to-18-month window | 0.034 | 2.306 | 0.044 | 3.467 |
| Baseline, all-stock cap | 0.035 | 2.321 | 0.048 | 4.084 |

```
  sorting_method = "univariate",
  rebalancing_month = 7,
  portfolio_sort_options = options_baseline
) |>
  compute_long_short_returns(direction = "bottom_minus_top")

factor_returns_all_cap <- implement_portfolio_sort(
  data = sorting_data,
  sorting_variables = "asset_growth",
  sorting_method = "univariate",
  rebalancing_month = 7,
  portfolio_sort_options = options_baseline,
  cap_weight_exchanges = NULL
) |>
  compute_long_short_returns(direction = "bottom_minus_top")
```

Table 5 decomposes the customized premium into two layers. Adding the size-controlled bivariate sort alone (the bivariate-only row) barely moves the annualized premium, from 3.5% to 3.6%, but raises the $t$-statistic from 2.32 to 3.06, because the size-controlled spread is less volatile. Layering the filter set and the 20% breakpoint threshold on top lowers the premium to 2.6% with a $t$-statistic of 1.89, which is no longer significant at the 5% level: removing financials, utilities, and loss-making firms and computing breakpoints without the smallest 20% of NYSE firms weakens the spread. In the sensitivity checks, the plain lag window gives a value-weighted premium of 3.4% ($t$-statistic of 2.31) instead of 3.5%, and capping at the all-stock percentile changes the capped value-weighted premium from 4.3% ($t$-statistic of 3.45) to 4.8% ($t$-statistic of 4.08). Across the three main specifications, the spread has the sign of the negative AG premium in Cooper et al. (2008), but is considerably smaller than their annual value-weighted long-short premium of about 12.6% over July 1968 to June 2003, which is based on deciles rather than quintiles.[2]

The lower-level functions in Table 1 share their arguments with `implement_portfolio_sort()`, so researchers can unroll the wrapper when custom logic applies at one stage only, for example, breakpoints computed within each industry or from a trailing window.

### Staggered rebalancing

Some factors are built with *staggered* rebalancing instead of a single annual or monthly formation date: portfolios are formed every month, and each cohort is held for $K$ months, so that $K$ overlapping cohorts are held at any time. Jegadeesh and Titman (1993) construct momentum this way. Their $J/K$ strategies sort stocks on the past $J$-month return and hold each cohort for $K$ months. In **tidyfinance**, the `holding_period` argument sets $K$. We compute the six-month momentum signal on calendar months and lag it by one month, so that the signal of month $t$ uses returns up to month $t-1$ only.

```
momentum_data <- crsp_monthly |>
  arrange(permno, date) |>
```

[2] Cooper et al. (2008) sorts into deciles.

**Table 6:** Long-short momentum returns with staggered rebalancing. Stocks are sorted into deciles on their lagged six-month return ($J = 6$) using NYSE breakpoints, and each monthly cohort is held for $K$ months; $K = 1$ is monthly rebalancing. The long-short portfolio buys the top decile (winners) and sells the bottom decile (losers). Means are annualized equal-weighted (EW) and value-weighted (VW) excess returns. Sample: 1972–2024.

| Holding period $K$ | EW mean | EW $t$ | VW mean | VW $t$ |
|---|---|---|---|---|
| 1 | -0.010 | -0.314 | 0.035 | 0.995 |
| 3 | 0.051 | 1.719 | 0.069 | 2.139 |
| 6 | 0.069 | 2.514 | 0.088 | 3.095 |
| 12 | 0.044 | 2.002 | 0.068 | 2.958 |

```
  group_by(permno) |>
  mutate(
    momentum = slider::slide_index_dbl(
      ret,
      date,
      \(x) if (length(x) == 6) prod(1 + x) - 1 else NA_real_,
      .before = months(5)
    )
  ) |>
  ungroup() |>
  add_lagged_columns(cols = "momentum", lag = months(1), by = "permno")
```

We then form decile portfolios on NYSE breakpoints and vary the holding period over $K \in \{1, 3, 6, 12\}$ months, where $K = 1$ is plain monthly rebalancing.

```
options_momentum <- portfolio_sort_options(
  breakpoint_options_main = breakpoint_options(
    n_portfolios = 10,
    breakpoints_exchanges = "NYSE"
  )
)
momentum_returns <- tibble(holding_period = c(1, 3, 6, 12)) |>
  mutate(
    factor_returns = purrr::map(
      holding_period,
      \(k) {
        implement_portfolio_sort(
          data = momentum_data,
          sorting_variables = "momentum_lag",
          sorting_method = "univariate",
          holding_period = k,
          portfolio_sort_options = options_momentum,
          quiet = TRUE
        ) |>
          compute_long_short_returns(direction = "top_minus_bottom")
      }
    )
  )
```

Table 6 shows that the holding period matters. With monthly rebalancing ($K = 1$), the equal-weighted momentum spread is -1.0% per year ($t$-statistic of -0.31). Holding six overlapping cohorts, as in the 6/6 strategy of Jegadeesh and Titman (1993), raises it to 6.9% ($t$-statistic of 2.51), and the value-weighted spread to 8.8% ($t$-statistic of 3.10). The signal, the breakpoints, and the rest of the call are identical across rows; only `holding_period` changes.

### Asset pricing tests

A long-short return is one summary of whether asset growth predicts the cross-section of returns. A complementary view comes from Fama-MacBeth regressions (Fama and MacBeth, 1973), which

**Table 7:** Fama-MacBeth regression of monthly excess returns on lagged asset growth and lagged log market capitalization. Coefficients are time-series averages of monthly cross-sectional OLS estimates; standard errors are Newey and West (1987) with first-order vector autoregressive prewhitening and the automatic bandwidth of Newey and West (1994). The intercept and slopes on the right-hand-side variables are reported on the monthly scale. Sample: 1974–2024.

| Variable | Risk premium | SE | *t* statistic | N (months) |
|---|---|---|---|---|
| Intercept | 0.0143 | 0.0046 | 3.1288 | 606 |
| Asset growth | -0.0046 | 0.0008 | -6.0417 | 606 |
| Log market cap | -0.0011 | 0.0004 | -2.4915 | 606 |

estimate the average risk premium associated with a characteristic after running cross-sectional regressions period by period. **tidyfinance** implements the standard procedure in `estimate_fama_macbeth()`, which takes a panel and a model formula, fits one cross-sectional ordinary least squares (OLS) regression per period, and aggregates the period-level coefficients into average risk premia with Newey and West (1987) standard errors. By default, these are the standard errors of `NeweyWest()` from **sandwich** (Zeileis, 2004): the time series of coefficients is prewhitened with a first-order vector autoregression, and the bandwidth is selected automatically following Newey and West (1994).

Here, we winsorize the AG signal at the 1st and 99th percentiles to keep extreme observations from dominating the cross-sectional regressions and pair it with log market capitalization as a size control.

```
fm_data <- sorting_data |>
  drop_na(asset_growth, mktcap_lag, ret_excess) |>
  group_by(date) |>
  mutate(
    asset_growth = winsorize(x = asset_growth, cut = 0.01),
    log_mktcap = log(mktcap_lag)
  ) |>
  ungroup()

fm_results <- estimate_fama_macbeth(
  data = fm_data,
  model = "ret_excess ~ asset_growth + log_mktcap"
)
```

The negative slope on `asset_growth` in Table 7 lines up with the long-short evidence in Table 4 and Table 5: high-AG firms earn lower subsequent returns, conditional on size. As an external benchmark, Cooper et al. (2008, Table III, Panel A, Model 1) reports a Fama-MacBeth slope on annual asset growth of -0.0922 (*t*-statistic of -6.52). Dividing by 12 gives a rough monthly counterpart of -0.0077 ($= -0.0922/12$). The comparison is only indicative, because Cooper et al. (2008) regress annual returns and include additional controls.[3] Switching to `vcov = "iid"` or supplying `vcov_options = list(lag = 6, prewhite = FALSE)` would change the standard-error treatment without altering the rest of the call, and adding additional regressors (e.g., a beta estimate from `estimate_betas()`, a profitability measure, an idiosyncratic-volatility control) is a one-line extension to the model formula. The same function therefore covers the standard cross-sectional asset-pricing test that complements the portfolio-sort evidence used throughout the rest of the paper.

## 5 Summary and discussion

Empirical asset pricing depends on dozens of construction choices that no consensus pins down. **tidyfinance** makes those choices first-class objects in the workflow: each is an argument, each has a documented default, each can be varied without rebuilding the pipeline. Disagreements over construction become empirical questions, not implicit assumptions buried in code.

We maintain and extend the package on CRAN. Unit tests cover all functions, with 100% code coverage reported by Codecov, and each release passes R CMD check on the latest macOS, Windows, and Ubuntu. A scheduled GitHub Actions workflow refreshes the companion risk-free rate on Hugging Face.

[3] Cooper et al. (2008) additionally control for book-to-market and 6- and 36-month past returns.

Companion materials at tidy-finance.org use **tidyfinance** in open coursework for graduate finance and asset pricing in R. The site also hosts a reviewed channel through which researchers can publish replications and new construction routines, so the catalog of factors and conventions can grow with the literature.

## Computational details

We obtained the results in this paper using R 4.6.1 with the following packages: **tidyfinance** 0.9.0.9007, **dplyr** 1.2.1 (Wickham et al., 2026), **tidyr** 1.3.2 (Wickham et al., 2024), **ggplot2** 4.0.3 (Wickham, 2016), and **lubridate** 1.9.5 (Spinu et al., 2024). We install development versions of **tidyfinance** with **remotes** 2.5.0 (Csárdi et al., 2024). Figures additionally use **scales** 1.4.0 (Wickham et al., 2023) and **patchwork** 1.3.2 (Pedersen, 2024). CRAN hosts R and all packages at `https://CRAN.R-project.org/`. **renv** 1.2.3 (Ushey and Wickham, 2026) ensures full reproducibility of the computational environment. Reproducing the factor return data requires an institutional subscription to WRDS, as the underlying CRSP and Compustat data are proprietary and cannot be redistributed.

Calling `rmarkdown::render("tidyfinance.Rmd")` renders the manuscript with WRDS data or pseudo data (Section 2.4.1). In the last render, running its code took 11 minutes or less. Timings reported here are obtained on Windows 11 x64 (build 26200) with a 8-core processor running R 4.6.1; WRDS extracts are not pre-cached, so this figure includes the time to download the underlying CRSP and Compustat panels.

We publish all code, data workflows, and documentation at `https://github.com/tidy-finance/paper-factor-construction`. The code is released under the MIT License, which is compatible with the GNU General Public License (GPL).

## Acknowledgments

We thank the editor, Emi Tanaka, and two anonymous referees for their thoughtful comments. Stefan Voigt is grateful to the Danish Finance Institute for financial support.

## References

P. Akey, A. Z. Robertson, and M. Simutin. Noisy Factors? The Retroactive Impact of Methodological Changes on the Fama–French Factors. *Review of Finance*, 30(5):1437–1485, 2026. doi: 10.1093/rof/rfag002. [p2]

N. Areal. **frenchdata***: Download Data Sets from Kenneth's French Finance Data Library Site*, 2021. R package version 0.2.0. [p2]

W. Armstrong, D. Eddelbuettel, J. Laing, and J. Ooms. **Rblpapi***: R Interface to 'Bloomberg'*, 2024. [p2]

T. G. Bali, R. F. Engle, and S. Murray. *Empirical Asset Pricing: The Cross Section of Stock Returns*. John Wiley & Sons, 2016. [p5, 7]

K. Boudt, J. Cornelissen, S. Payseur, O. Kleen, and E. Sjoerup. **highfrequency***: Tools for Highfrequency Data Analysis*, 2024. [p2]

A. Chen and T. Zimmermann. Open Source Cross-Sectional Asset Pricing. *Critical Finance Review*, 11 (2):207–264, 2022. doi: 10.1561/104.00000112. [p1, 2]

M. J. Cooper, H. Gulen, and M. J. Schill. Asset Growth and the Cross-Section of Stock Returns. *The Journal of Finance*, 63(4):1609–1651, 2008. doi: 10.1111/j.1540-6261.2008.01370.x. [p1, 9, 11, 13, 15, 17]

G. Csárdi, J. Hester, H. Wickham, W. Chang, M. Morgan, and D. Tenenbaum. **remotes***: R Package Installation from Remote Repositories, Including 'GitHub'*, 2024. R package version 2.5.0. [p18]

A. Damodaran. Damodaran online: Data, 2026. URL `https://pages.stern.nyu.edu/~adamodar/New_Home_Page/datacurrent.html`. [p4]

M. Dancho and D. Vaughan. **tidyquant***: Tidy Quantitative Financial Analysis*, 2026. R package version 1.0.12. [p2]

J. Dick-Nielsen. Liquidity Biases in TRACE. *The Journal of Fixed Income*, 19(2):43–55, 2009. doi: 10.3905/jfi.2009.19.2.043. [p5, 6]

A. Dickerson, P. Mueller, and C. Robotti. Priced Risk in Corporate Bonds. *Journal of Financial Economics*, 150(2):103707, 2023. doi: 10.1016/j.jfineco.2023.103707. [p1, 2]

D. Eddelbuettel, K. Nguyen, and T. Leitch. **RQuantLib***: R Interface to the 'QuantLib' Library*, 2024. [p2]

E. F. Fama and K. R. French. The Cross-Section of Expected Stock Returns. *The Journal of Finance*, 47(2): 427–465, 1992. doi: 10.1111/j.1540-6261.1992.tb04398.x. [p5]

E. F. Fama and K. R. French. Common Risk Factors in the Returns on Stocks and Bonds. *Journal of Financial Economics*, 33(1):3–56, 1993. doi: 10.1016/0304-405X(93)90023-5. [p2, 6, 7, 10]

E. F. Fama and K. R. French. Production of U.S. Rm-Rf, SMB, and HML in the Fama-French Data Library. Chicago Booth Research Paper No. 23-22, Fama-Miller Working Paper, Dec. 2023. [p1, 2]

E. F. Fama and J. D. MacBeth. Risk, Return, and Equilibrium: Empirical Tests. *Journal of Political Economy*, 81(3):607–636, 1973. doi: 10.1086/260061. [p16]

N. Hautsch, A. J. Menkveld, and S. Voigt. Market responses to a VIX shock. Working Paper, 2026. [p5]

K. Hou, C. Xue, and L. Zhang. Replicating Anomalies. *The Review of Financial Studies*, 33(5):2019–2133, 2020. doi: 10.1093/rfs/hhy131. [p2]

K. Hou, H. Mo, C. Xue, and L. Zhang. An Augmented Q-Factor Model with Expected Growth. *Review of Finance*, 25(1):1–41, 2021. doi: 10.1093/rof/rfaa004. [p4]

N. Jegadeesh and S. Titman. Returns to Buying Winners and Selling Losers: Implications for Stock Market Efficiency. *The Journal of Finance*, 48(1):65–91, 1993. doi: 10.1111/j.1540-6261.1993.tb04702.x. [p7, 15, 16]

T. I. Jensen, B. Kelly, and L. H. Pedersen. Is There a Replication Crisis in Finance? *The Journal of Finance*, 78(5):2465–2518, 2023. doi: 10.1111/jofi.13249. [p1, 2, 4, 7]

M. W. McCracken and S. Ng. FRED-MD: A Monthly Database for Macroeconomic Research. *Journal of Business & Economic Statistics*, 34(4):574–589, 2016. doi: 10.1080/07350015.2015.1086655. [p4]

M. W. McCracken and S. Ng. FRED-QD: A Quarterly Database for Macroeconomic Research. *Federal Reserve Bank of St. Louis Review*, 103(1):1–44, 2021. doi: 10.20955/r.103.1-44. [p4]

W. K. Newey and K. D. West. A Simple, Positive Semi-Definite, Heteroskedasticity and Autocorrelation Consistent Covariance Matrix. *Econometrica*, 55(3):703–708, 1987. doi: 10.2307/1913610. [p4, 17]

W. K. Newey and K. D. West. Automatic Lag Selection in Covariance Matrix Estimation. *The Review of Economic Studies*, 61(4):631–653, 1994. doi: 10.2307/2297912. [p4, 17]

Ľ. Pástor and R. F. Stambaugh. Liquidity Risk and Expected Stock Returns. *Journal of Political Economy*, 111(3):642–685, 2003. doi: 10.1086/374184. [p4]

T. L. Pedersen. **patchwork***: The Composer of Plots*, 2024. R package version 1.3.2. [p18]

B. G. Peterson and P. Carl. **PerformanceAnalytics***: Econometric Tools for Performance and Risk Analysis*, 2026. R package version 2.1.0. [p2]

G. Rossetti and A. Dickerson. **PyBondLab***: Portfolio Sorting and Strategy Evaluation for Corporate Bonds*, 2026. URL https://github.com/GiulioRossetti94/PyBondLab. Python package version 0.3.0. [p1, 2]

J. A. Ryan and J. M. Ulrich. **quantmod***: Quantitative Financial Modelling Framework*, 2025. R package version 0.4.28. [p2]

C. Scheuch, S. Voigt, and P. Weiss. *Tidy Finance with R*. Chapman and Hall/CRC, 1st edition, 2023. doi: 10.1201/b23237. [p1, 2, 5]

P. Schwarz, D. Walter, and P. Weiss. Rewriting CRSP's History: Impact of Altered Monthly Returns on Asset Pricing. *Journal of Financial and Quantitative Analysis*, page 1–37, 2026. doi: 10.1017/S0022109026102774. [p5]

A. Soebhag, B. Van Vliet, and P. Verwijmeren. Non-standard errors in asset pricing: Mind your sorts. *Journal of Empirical Finance*, 78:101517, 2024. doi: 10.1016/j.jempfin.2024.101517. [p1]

V. Spinu, G. Grolemund, and H. Wickham. **lubridate***: Make Dealing with Dates a Little Easier*, 2024. URL https://lubridate.tidyverse.org. R package version 1.9.5. [p8, 18]

R. F. Stambaugh and Y. Yuan. Mispricing Factors. *The Review of Financial Studies*, 30(4):1270–1315, 2017. doi: 10.1093/rfs/hhw107. [p4]

K. Ushey and H. Wickham. **renv***: Project Environments*, 2026. R package version 1.2.3. [p18]

D. Walter, R. Weber, and P. Weiss. Methodological uncertainty in portfolio sorts. Working Paper, 2024. [p1]

I. Welch and A. Goyal. A Comprehensive Look at the Empirical Performance of Equity Premium Prediction. *The Review of Financial Studies*, 21(4):1455–1508, 2008. doi: 10.1093/rfs/hhm014. [p5]

H. Wickham. **ggplot2***: Elegant Graphics for Data Analysis*. Springer-Verlag, New York, 2016. ISBN 978-3-319-24277-4. URL https://ggplot2.tidyverse.org. [p18]

H. Wickham. The tidy tools manifesto, Nov. 2023. URL https://tidyverse.tidyverse.org/articles/manifesto.html. Accessed: 2026-06-05. [p2]

H. Wickham, T. L. Pedersen, and D. Seidel. **scales***: Scale Functions for Visualization*, 2023. R package version 1.4.0. [p18]

H. Wickham, D. Vaughan, and M. Girlich. **tidyr***: Tidy Messy Data*, 2024. URL https://tidyr.tidyverse.org. R package version 1.3.2. [p8, 18]

H. Wickham, R. François, L. Henry, K. Müller, and D. Vaughan. **dplyr***: A Grammar of Data Manipulation*, 2026. URL https://dplyr.tidyverse.org. R package version 1.2.1. [p8, 18]

D. Wuertz, T. Setz, Y. Chalabi, and W. Chen. **fPortfolio***: Rmetrics — Portfolio Selection and Optimization*, 2023. [p2]

A. Zeileis. Econometric Computing with HC and HAC Covariance Matrix Estimators. *Journal of Statistical Software*, 11(10):1–17, 2004. doi: 10.18637/jss.v011.i10. [p17]

*Christoph Frey*
*Lancaster University*
*Management School, Bailrigg*
*Lancaster LA1 4YX, United Kingdom*
*ORCiD: 0000-0002-9937-0903*
christoph.frey@gmail.com

*Christoph Scheuch*
*Humboldt University of Berlin*
*School of Business and Economics, Dorotheenstraße 1*
*10099 Berlin, Germany*
*ORCiD: 0009-0004-0423-6819*
christoph.scheuch@hu-berlin.de

*Stefan Voigt*
*University of Copenhagen, Danish Finance Institute*
*Department of Economics, Øster Farimagsgade 5*
*1353 Copenhagen K, Denmark*
*ORCiD: 0000-0001-5619-3161*
stefan.voigt@econ.ku.dk

*Patrick Weiss*
*Reykjavík University*
*Department of Business and Economics, Menntavegur 1*
*102 Reykjavík, Iceland*
*ORCiD: 0000-0002-9282-5872*
patrickw@ru.is